

Dictionaries

03 August 2026 18:12

The data type dictionary fall under mapping. It is a mapping between a *set of keys* and a *set of values*. The key - value pair is called an item. A key is separated from its value by (:) and consecutive items are separated by commas. Items and dictionaries are unordered, so we may not get back the data in the same order in which we had entered the data initially in the dictionary.

Creating Dictionary

To create a dictionary. The items entered are separated by commas and enclosed in curly braces. Each item is a key value pair separated through colon(:). The keys in the dictionary must be unique and should be of any immutable data type, that is number, string or a tuple. The values can be repeated and can be of any data type.

```
>>> dict1 = {} ← empty dictionary initialization
>>> type(dict1)
<class 'dict'>
>>> dict2 = dict()
>>> type(dict2)
<class 'dict'>
>>> dict1
{}
>>> dict2
{}
>>> dict3 = {'Mohan':95, 'Ram':89, 'Suhel':92, 'Sangeeta':85}
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 92, 'Sangeeta': 85}
```

Accessing items in a dictionary

We have already seen that the items of a sequence (string, list and tuple) are accessed using a technique called indexing. The items of a dictionary are accessed via the keys rather than via their relative positions or indices. Each key serves as the index and maps to a value. The following example shows how a dictionary returns the value corresponding to the given key:

```
>>> dict3 = {'Mohan':95, 'Ram':89, 'Suhel':92, 'Sangeeta':85}
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 92, 'Sangeeta': 85}
>>> dict3['Sangeeta']
85
>>> dict3['Shyam']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

```
dict3['Shyam']
~~~~^^^^^^^^^^
KeyError: 'Shyam'
>>> dict3['Ram']
89
>>>
```

Dictionaries are mutable

Dictionaries are mutable, which implies that the content of the dictionary can be changed after it has been created.

Adding a new item

We can add new item to the dictionary as shown in the following example:

```
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 92, 'Sangeeta': 85}
>>> dict3['Shiv'] = 99
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 92, 'Sangeeta': 85, 'Shiv': 99}
```

Modifying an Existing Item

The existing dictionary can be modified by just overwriting the key value pair. Example two, modify a given item in the dictionary:

```
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 92, 'Sangeeta': 85, 'Shiv': 99}
>>> dict3['Suhel'] = 93.5
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99}
```

Dictionary operations

Membership

The membership operator *in* checks if the key is present in the dictionary and returns *True*, else it returns *False*.

```
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99}
>>> 'Shiv' in dict3
True
>>> 'Shyam' in dict3
False
```

The not in operator returns *True* if the key is not present in the dictionary, else it returns *False*.

```
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99}
>>> 'Suhel' not in dict3
False
>>> 'Shyam' not in dict3
True
```

Traversing A Dictionary

We can access each item of the dictionary or traverse a dictionary using for loop.

Method-1:

```
>>> dict3
{'Mohan': 95, 'Ram': 89, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99}
>>> for key in dict3:
...     print(f'{key} : {dict3[key]}')
...
Mohan : 95
Ram : 89
Suhel : 93.5
Sangeeta : 85
Shiv : 99
```

Method-2:

```
>>> for key, value in dict3.items():
...     print(f'{key} : {value}')
...
Mohan : 95
Ram : 89
Suhel : 93.5
Sangeeta : 85
Shiv : 99
```

Dictionary methods and built-in functions

Python provides many functions to work on dictionaries. Here is a list of some of the commonly used dictionary methods.

Built-in functions and methods for dictionary.

Method	Description	Example
<code>len()</code>	Returns the length or number of <i>keys : value</i> pairs of the dictionary passed as the argument.	<pre>>>> dict3 {'Mohan': 95, 'Ram': 89, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99} >>> len(dict3)</pre>

		5
dict()	Create a dictionary. A sequence of key - value pairs.	<pre> >>> pair = [('Jack', 95), ('Amy', 88), ('Jhon', 79), ('Omi', 87)] >>> pair [('Jack', 95), ('Amy', 88), ('Jhon', 79), ('Omi', 87)] >>> dict1 = dict(pair) >>> dict1 {'Jack': 95, 'Amy': 88, 'Jhon': 79, 'Omi': 87} >>> dict1['Amy'] 88 </pre>
keys()	Returns a list of keys in the dictionary.	<pre> >>> dict1.keys() dict_keys(['Jack', 'Amy', 'Jhon', 'Omi']) >>> dict1_keys = dict1.keys() >>> dict1_keys dict_keys(['Jack', 'Amy', 'Jhon', 'Omi']) >>> type(dict1_keys) <class 'dict_keys'> </pre>
values()	Return a list of values in the dictionary.	<pre> >>> dict1.values() dict_values([95, 88, 79, 87]) </pre>
items()	Returns a list of tuples (key-value) pair.	<pre> >>> dict1.items() dict_items([('Jack', 95), ('Amy', 88), ('Jhon', 79), ('Omi', 87)]) </pre>
get()	Returns the value corresponding to the key passed as the argument. If the key is not present in the dictionary, it will return None.	<pre> >>> dict1 {'Jack': 95, 'Amy': 88, 'Jhon': 79, 'Omi': 87} >>> dict1.get('Jhon') 79 >>> dict1.get('Omi') 87 >>> dict1.get('Shiv') >>> val = dict1.get('Shiv') >>> val >>> type(val) <class 'NoneType'> </pre>
update()	Appends the key - value pair of the dictionary passed as the argument to the key - value pair of the given dictionary.	<pre> >>> dict1 {'Jack': 95, 'Amy': 88, 'Jhon': 79, 'Omi': 87} >>> dict1.update(dict3) >>> dict1 {'Jack': 95, 'Amy': 88, </pre>

		<pre>'Jhon': 79, 'Omi': 87, 'Mohan': 95, 'Ram': 89, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99} >>></pre>
del()	Deletes the item with the given key. To delete the dictionary from the memory we write: <pre>del Dict_name</pre>	<pre>>>> dict3 {'Mohan': 95, 'Ram': 89, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99} >>> del dict3['Ram'] >>> dict3 {'Mohan': 95, 'Suhel': 93.5, 'Sangeeta': 85, 'Shiv': 99} >>> del dict3['Shiv'] >>> dict3 {'Mohan': 95, 'Suhel': 93.5, 'Sangeeta': 85} >>> del dict3 >>> dict3 Traceback (most recent call last): File "<stdin>", line 1, in <module> dict3 NameError: name 'dict3' is not defined. Did you mean: 'dict1'?</pre>
clear()	Deletes or clear all the items of the dictionary.	<pre>>>> pair = [('Jack', 95), ('Amy', 88), ('Jhon', 79), ('Omi', 87)] >>> dict3 = dict(pair) >>> dict3 {'Jack': 95, 'Amy': 88, 'Jhon': 79, 'Omi': 87} >>> dict3.clear() >>> dict3 {}</pre>

Manipulating Dictionaries

Q: Create a dictionary of. Odd numbers between one and 10. Where the key is the decimal number and the value is the corresponding number in words. Perform the following operations on this dictionary:

- i. Display the keys.
- ii. Display the values.

- iii. Display the items.
- iv. Find the length of the dictionary.
- v. Check if 7 is present or not.
- vi. Check if two is present or not.
- vii. Retrieve the value corresponding to the key 9.
- viii. Delete the item from the dictionary corresponding to key 9.

```
ODD = {1 : 'One', 3 : 'Three', 5 : 'Five', 7 : 'Seven', 9 : 'Nine'}
print(ODD)
print(f'Keys: {ODD.keys()}')
print(f'Values: {ODD.values()}')
print(f'Items: {ODD.items()}')
print(f'Number of items: {len(ODD)}')
print(f'Is 7 present: {7 in ODD}')
print(f'Is 2 present: {2 in ODD}')
print(f'9 : {ODD[9]}')
del ODD[9]
print(f'After deletion: {ODD}')
```

Output:

```
{1: 'One', 3: 'Three', 5: 'Five', 7: 'Seven', 9: 'Nine'}
Keys: dict_keys([1, 3, 5, 7, 9])
Values: dict_values(['One', 'Three', 'Five', 'Seven', 'Nine'])
Items: dict_items([(1, 'One'), (3, 'Three'), (5, 'Five'), (7, 'Seven'), (9, 'Nine')])
Number of items: 5
Is 7 present: True
Is 2 present: False
9 : Nine
After deletion: {1: 'One', 3: 'Three', 5: 'Five', 7: 'Seven'}
```

Homework:

1. Write a program to enter names of employees and their salaries as input and store them in a dictionary.
2. Write a program to count the number of times a character appears in a given string.
3. Write a function to convert a number entered by the user into its corresponding number in words. For example, if the input is 876, then the output should be 'Eight Seven Six'.
4. Write a program to input your friends name and their phone numbers and store them in the dictionary as a key value pair. Perform the following operations on the dictionary:
 - i. Display the name and phone numbers of all your friends.
 - ii. Add a new key value pair in this dictionary and display the modified dictionary.

- iii. Delete a particular friend from the dictionary.
- iv. Modify the phone number of an existing friend.
- v. Check if a friend is present in the dictionary or not.
- vi. Display the dictionary in sorted order of names.